

Pinpointing Interesting Code in Binaries: Heuristics, Statistics, and Agents

Binary Cartography

Tim Blazytko



@mr_phrazer



synthesis.to



tim@blazytko.to

About Tim

- independent security researcher, consultant & trainer
- PhD in binary program analysis & reverse engineering
- focus on (de)obfuscation & automation
- trainings on reverse engineering & software deobfuscation



 Navigating Large, Unknown Binaries

 Heuristics & Statistical Signals

 Heuristics Meet Agents

❓ Large, Unknown Binary

Where to Start?

- function symbols
- meaningful strings
- interesting API functions & cross-references

Analysis Challenges

- locating **complex state machines** and protocol logic
- detecting **cryptographic implementations**
- discovering **C&C server communication** and **string decryption routines**
- pinpointing **obfuscated code** in commercial applications
- identifying **API functions** in **statically-linked executables**

→ Code Detection Heuristics

Identification of **interesting** code constructs

- **guide** manual analysis
- architecture-agnostic
- efficient to compute

How?

Code Complexity and Statistical Analysis

Interesting code is

- (artificially) complex
 - frequently executed
 - uncommon
-
- basic block/function size
 - control-flow graph characteristics
 - frequency analysis
 - usage of intermediate representations

Detection Heuristics

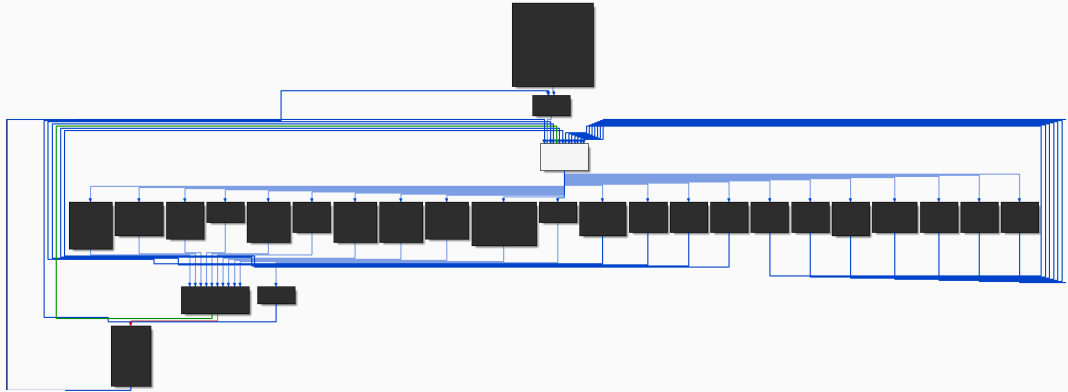
Heuristics

1. state machines
2. complex functions
3. large basic blocks
4. frequently called functions
5. cryptographic & string decryption routines

- most heuristics **relative** to all functions in the binary
- each heuristic detects **different** patterns

Identification of State Machines

State Machine Heuristic



Identification of functions with **loop-based** dispatching routines

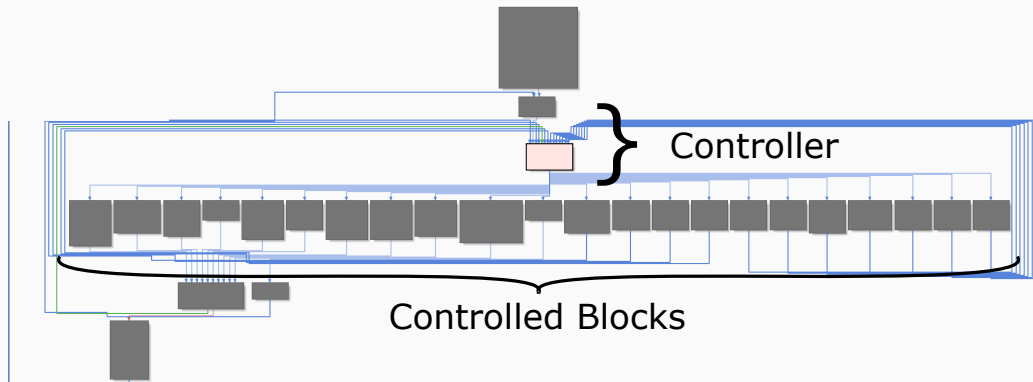
```
while(true) {  
    switch(state) {  
        case state_0: ...  
        case state_1: ...  
        ...  
        case state_n: ...  
    }  
}
```

- state machines often implement a **complex program logic**

Complex State Machines

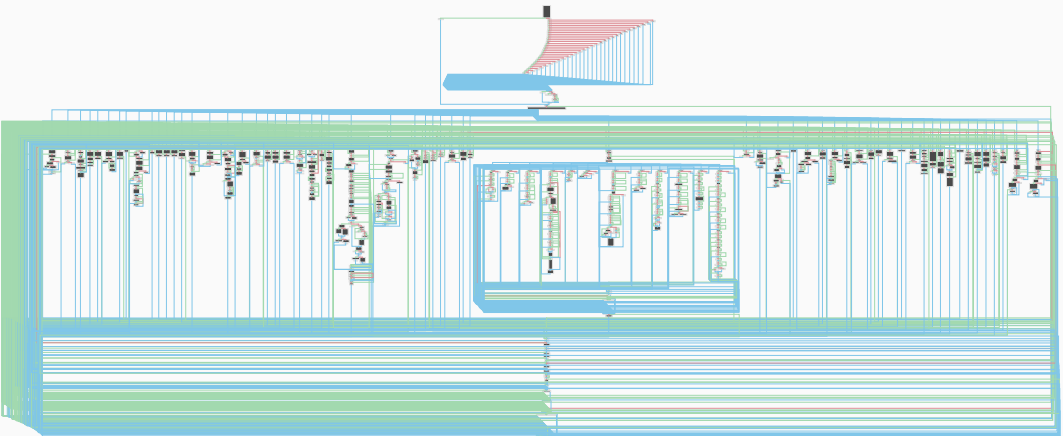
- file format parsing
- input validation & sanitization
- network protocol dispatching
- C&C server communication & command dispatching
- data encoding/decoding

State Machine Heuristic



$$\frac{\text{\#controlled blocks}}{\text{\#blocks in the function}}$$

Complex Functions

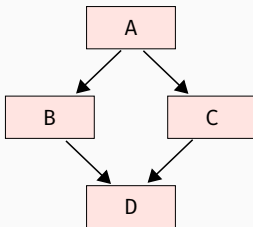


Identification of functions with large control-flow graphs

- large functions indicate a **complex code logic**
 - file parsing
 - dispatching routines and network protocols
 - obfuscation
- efficient metric: **cyclomatic complexity**

Cyclomatic Complexity

$$\#edges - \#basic\ blocks + 2$$



- 4 basic blocks
- 4 edges

cyclomatic complexity: 2

Large Basic Blocks



Identification of functions with large basic blocks

- ~5-7 instructions per basic block (on average)
- larger basic blocks indicate **complex straight-line code**
- compute per function:

$$\frac{\text{\#instructions}}{\text{\#basic blocks}}$$

Complex Straight-Line Code

- unrolled loops
- cryptographic implementations
- initialization routines
- arithmetic obfuscation

Frequently Called Functions

[illegible]

Frequently Called Functions

Identification of functions which are **frequently called** from **different functions**

- allows the identification of **API functions** in **statically-linked executables**
- can sometimes also detect **string decryption & hash functions** in malware

Frequently Called API Functions

- memory management
- data movement
- string operations
- file I/O operations

XOR Decryption Loops

XOR Decryption Loops

Identification of loops that **XOR** data against a constant

```
void xor_decrypt(char *input, int length) {  
    for (int i = 0; i < length; i++) {  
        input[i] = input[i] ^ 0xde;  
    }  
}
```

- string decryption and code unpacking

RC4

Identification of the RC4 key-scheduling algorithm (KSA)

```
void ksa(uint8_t *key, uint8_t *S, int key_length) {  
    for (int i = 0; i < 256; ++i) { ... }  
  
    int j = 0;  
    for (int i = 0; i < 256; ++i) { ... }  
}
```

- two loops with the constant 256
- most-common crypto in malware — string decryption, DGA



From Heuristics to Agents

Heuristics as a Triage Signal

Heuristics **rank** — agents **read, reason, and verify**

- cheap, architecture-agnostic, over the whole binary
- output: a ranked list of suspicious functions
- a triage signal — not proof

Pruning False Positives

Heuristics flag **structure**, not **meaning**

- noise: compiler output, libc code, parsers, big switches
- the agent reads the function and confirms or discards it
- cross-correlate: many hits on one function = strong lead

Heuristics pinpoint **structural patterns**

- skip initialization, glue, and library code
- start at state machines, crypto, and C&C dispatch
- deeper analysis within the context window

- **obfuscation_detection** — Binary Ninja plugin
- **obfuscation_detection_ghidra** — Ghidra extension
- GUI tools that also run headless (**run_detections**)
- agents run the detections, then decompile the top findings

https://github.com/mrphrazer/obfuscation_detection

https://github.com/mrphrazer/obfuscation_detection_ghidra

Wrap-Up

- heuristics over-approximate — expect false positives
- scores are relative to the binary
- agent results are non-deterministic — verify

Summary

- heuristics & statistics surface interesting code
- state machines, complexity, large blocks, most-called, RC4
- agents prune false positives and reach relevant code faster

<https://github.com/mrphrazer/binary-cartography>

Tim Blazytko



@mr_phrazer



synthesis.to



tim@blazytko.to

- **Reverse Engineering – Binary Program Analysis**
 - 4 days, hands-on, from fundamentals to advanced topics
- **Software Deobfuscation Techniques**
 - 4 days, advanced: systematic workflows for analyzing protected binaries
 - **Hexacon 2026**, Paris, October 12–15



<https://synthesis.to/trainings.html>

private trainings available on-site & remote — tim@blazytko.to